

DE L'INSTRUMENT ET DU NOMBRE

Les Ambassadeurs de Holbein posent leurs instruments de mesure du monde — globe, quadrant, cadran — sur un tapis richement chargé : R est ce même établi, réduit à un langage, où chaque fonction mesure, résume ou représente une donnée à la place d'un instrument de laiton.



FIG. VI. HANS HOLBEIN LE JEUNE, « LES AMBASSADEURS », 1533

Tutoriel complet du langage R : syntaxe, structures de données, visualisation avec ggplot2, statistique et cartographie spatiale, en trois parcours de niveau.

SOMMAIRE

1. STATISTIQUE SPATIALE AVEC SPDEP : RECALCULER MORAN EN R

2. RÉGRESSION SPATIALE AVEC SPATIALREG

3. GÉOSTATISTIQUE ET KRIGEAGE AVEC GSTAT

4. MODÈLES ADDITIFS GÉNÉRALISÉS POUR DONNÉES SPATIALES (MGCV)

5. SIMULATION MONTE CARLO, BOOTSTRAP SPATIAL ET TESTS DE PERMUTATION

6. REPRODUCTIBILITÉ : RENV, HERE, GIT

7. QUARTO ET R MARKDOWN AVANCÉ : RAPPORTS PARAMÉTRÉS

R est un langage de programmation et un environnement dédiés au calcul statistique et à la visualisation de données, né au début des années 1990 comme implémentation libre du langage S. En géomatique, R occupe une place particulière : au-delà de la statistique généraliste, il porte un écosystème de packages spatiaux mature (sf, terra, spdep, gstat, tmap) qui permet de reproduire en code, ligne par ligne, ce qu'un SIG comme QGIS fait par clics — avec l'avantage d'un script reproductible, versionnable et automatisable sur

des dizaines de jeux de données identiques. Ce module présente RStudio, la syntaxe du langage, la visualisation avec ggplot2, puis la cartographie et la statistique spatiale. Trois pistes complètes ci-dessous (choisis la tienne dans le filtre « Afficher ») : chacune se lit seule, du début à la fin.

APPROFONDISSEMENT

1. STATISTIQUE SPATIALE AVEC SPDEP : RECALCULER MORAN EN R

Le module Le Compas introduit l'indice de Moran global comme formule théorique ; le module Les Statistiques le décompose localement (LISA, G_i^*). Le package spdep implémente directement ces calculs en R, à partir d'une matrice de voisinage construite explicitement plutôt qu'implicite dans un logiciel graphique.

MATRICE DE VOISINAGE ET TEST DE MORAN

```
nb <- poly2nb(communes) lw <- nb2listw(nb, style = 'W')
moran.test(communes$taux, lw)
```

poly2nb() construit la liste de voisinage à partir des polygones de communes (contiguïté par contact de frontière, par défaut). nb2listw() la convertit en objet listw pondéré (style = 'W' : pondération normalisée par ligne, chaque voisin comptant 1/nombre de voisins). moran.test(x, listw) exécute le test paramétrique de significativité de l'indice de Moran, calculé sur la variable communes\$taux — la même formule $I = (n/S0) \cdot [\sum_i \sum_j w_{ij} (x_i - \bar{x})(x_j - \bar{x})] / \sum_i (x_i - \bar{x})^2$ que le module Le Compas présente en théorie.

ATTENTION

LE CHOIX DE LA MATRICE DE VOISINAGE N'EST PAS NEUTRE

poly2nb() accepte un argument queen (TRUE par défaut : contiguïté « reine », un simple contact de coin suffit ; FALSE : contiguïté « tour », il faut un partage de frontière) qui change la matrice de voisinage, et donc potentiellement le résultat du test — le même avertissement que le module Le Compas fait pour tout choix de pondération w_{ij} , à documenter systématiquement plutôt qu'à laisser à la valeur par défaut sans vérification.

VOIR AUSSI : REVOIR LA THÉORIE : INDICE DE MORAN ET MAUP

Le module Le Compas introduit la formule de Moran et le problème du zonage arbitraire (MAUP) que ce calcul R implémente directement.

VOIR AUSSI : REVOIR LA THÉORIE : LISA ET GI*

Le module Les Statistiques décompose Moran en indicateurs locaux (LISA) et détecte les points chauds (GI*) — `spdep` fournit `localmoran()` et `localG()` pour les recalculer en R de la même façon que `moran.test()` ci-dessus.

APPROFONDISSEMENT

2. RÉGRESSION SPATIALE AVEC SPATIALREG

Une régression classique (moindres carrés ordinaires, OLS) suppose des résidus indépendants — une hypothèse régulièrement violée sur donnée spatiale (module Les Statistiques). `spatialreg` ajuste des modèles qui intègrent explicitement l'autocorrélation spatiale : le modèle à retard spatial (SAR, spatial lag) et le modèle à erreur spatiale (souvent estimé comme un CAR, conditional autoregressive).

OLS CLASSIQUE, PUIS DIAGNOSTIC DES RÉSIDUS

```
modele_ols <- lm(taux ~ chomage + densite, data = communes)
lm.morantest(modele_ols, lw)
```

`lm()` ajuste une régression OLS ordinaire. `lm.morantest()` (`spdep`) calcule l'indice de Moran directement sur les résidus du modèle plutôt que sur la variable brute : un résultat significatif signale que l'OLS a laissé passer une structure spatiale qu'un modèle spatial expliciterait mieux.

MODÈLE À RETARD SPATIAL (SAR) ET À ERREUR SPATIALE (CAR)

```
modele_sar <- lagsarlm(taux ~ chomage + densite, data =
communes, listw = lw) modele_car <- spautolm(taux ~ chomage +
densite, data = communes, listw = lw, family = 'CAR')
summary(modele_sar)
```

`lagsarlm()` ajuste un modèle à retard spatial : la valeur d'une commune dépend explicitement de la moyenne pondérée de ses voisines, en plus des variables explicatives classiques (utile quand le phénomène se diffuse, une contagion). `spautolm(..., family = 'CAR')` ajuste un modèle conditionnel autorégressif, qui modélise plutôt la structure spatiale dans le terme d'erreur.

OLS (LM)

- Suppose des résidus indépendants
- Rapide, coefficients directement interprétables
- Sous-estime l'incertitude réelle si les résidus sont autocorrélés spatialement

SAR / CAR (SPATIALREG)

- Intègre explicitement l'autocorrélation, via `lw`
- Coefficients plus fiables, incertitude correctement estimée
- Interprétation plus subtile : un coefficient SAR inclut un effet de diffusion indirect vers les voisins

REMARQUE

GWR : RAPPEL DU MODULE LES STATISTIQUES

SAR et CAR corrigent l'autocorrélation mais gardent un coefficient unique pour tout le territoire. La régression géographiquement pondérée (GWR, module Les Statistiques) ajuste un coefficient différent en chaque point ; le package R correspondant, `spgwr`, en est l'implémentation directe — non détaillé ici pour rester concentré sur la correction de l'autocorrélation résiduelle plutôt que sur la variation spatiale des coefficients.

APPROFONDISSEMENT

3. GÉOSTATISTIQUE ET KRIGEAGE AVEC GSTAT

Le module Le Compas introduit le krigeage en théorie, comme méthode d'interpolation statistiquement optimale une fois le variogramme ajusté. Le package `gstat` en est l'implémentation R directe : de la même donnée ponctuelle aux mêmes trois paramètres (portée, palier, pépite), jusqu'à la carte krigée et sa carte d'incertitude — ce que QGIS ou TerrSet font via un module d'interpolation en interface, `gstat` le fait ligne de code par ligne de code, reproductible et scriptable sur autant de jeux de données que nécessaire.

VARIOGRAMME EXPÉRIMENTAL, PUIS AJUSTEMENT D'UN MODÈLE THÉORIQUE

```
v <- variogram(zinc ~ 1, data = mesures) v_modele <-  
fit.variogram(v, model = vgm(psill = 1, model = 'Sph', range =  
300, nugget = 0.1)) plot(v, v_modele)
```

`variogram(zinc ~ 1, data = mesures)` calcule le variogramme expérimental de la variable zinc sur l'objet spatial `mesures` (le membre droit `~ 1` signale une interpolation sans variable auxiliaire — un krigeage ordinaire). `vgm()` propose un modèle théorique de départ (ici sphérique, 'Sph', avec palier, portée et pépité estimés à l'œil sur le nuage expérimental) ; `fit.variogram()` en affine ensuite les paramètres par optimisation numérique. `plot(v, v_modele)` superpose le nuage expérimental et la courbe ajustée — la même figure que le diagramme théorique ci-dessous.

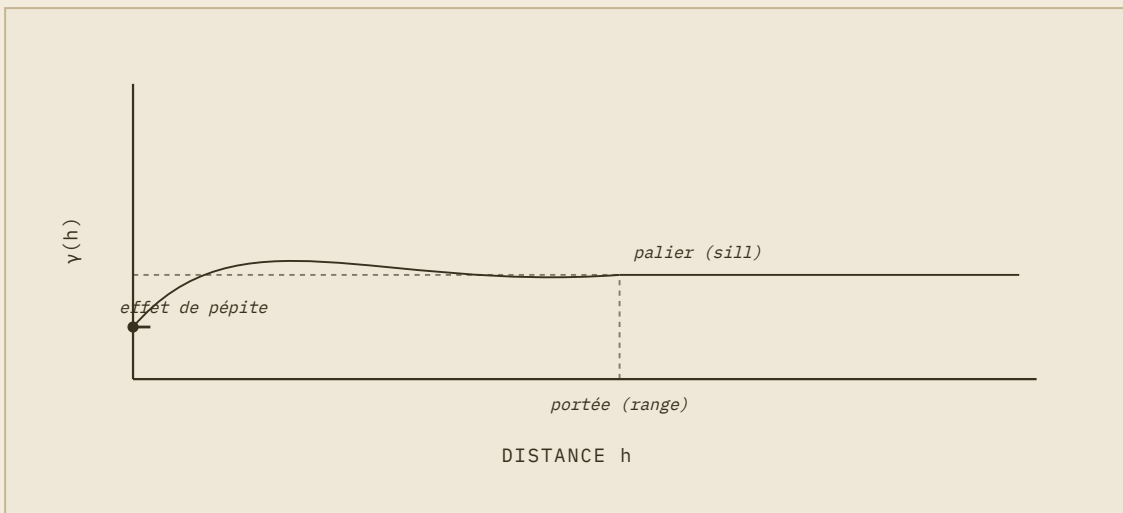


PLANCHE XV. FORME TYPIQUE D'UN VARIOGRAMME : EFFET DE PÉPITÉ À L'ORIGINE, MONTÉE JUSQU'AU PALIER, ATTEINT À LA PORTÉE — LES TROIS PARAMÈTRES QUE `FIT.VARIOGRAM()` ESTIME NUMÉRIQUEMENT.

KRIGEAGE ORDINAIRE SUR UNE GRILLE

```
krigeage <- krige(zinc ~ 1, locations = mesures, newdata =  
grille, model = v_modele)
```

`krige()` interpole la variable zinc sur chaque point/cellule de grille, en utilisant le modèle de variogramme `v_modele` ajusté ci-dessus. Le résultat comporte deux colonnes : `var1.pred` (la valeur interpolée) et `var1.var` (la variance de krigeage, la carte d'incertitude que ni le plus proche voisin ni l'IDW ne fournissent — le point déjà souligné dans le module `Le Compas`).

REMARQUE

REPRODUIRE EN CODE CE QU'UN MODULE GRAPHIQUE FAIT EN UN CLIC

L'interpolation par krigeage disponible dans QGIS (via SAGA ou GRASS) ou dans TerrSet propose les mêmes trois paramètres (pépite, palier, portée) derrière une boîte de dialogue. Le faire en R avec gstat n'apporte rien de statistiquement différent, mais rend chaque étape (ajustement du variogramme, choix du modèle théorique, paramètres retenus) explicite, documentée dans un script, et immédiatement reproductible sur un nouveau jeu de mesures sans reconfigurer une interface à chaque fois.

VOIR AUSSI : REVOIR LA THÉORIE : VARIOGRAMME, PORTÉE, PALIER, PÉPITE

Le module Le Compas introduit le krigeage et le variogramme théoriquement ; cette section en est le contrepoint direct, implémenté en R avec gstat.

APPROFONDISSEMENT

4. MODÈLES ADDITIFS GÉNÉRALISÉS POUR DONNÉES SPATIALES (MGCV)

Une régression linéaire (OLS, ou même SAR/CAR ci-dessus) suppose une relation linéaire entre variables explicatives et variable à expliquer. Quand cette hypothèse ne tient pas — un NDVI qui augmente avec l'altitude jusqu'à un optimum puis rechute, par exemple — un modèle additif généralisé (GAM, package mgcv) ajuste une courbe lisse plutôt qu'une droite, sans imposer sa forme à l'avance.

GAM AVEC LISSAGE SPATIAL

```
modele_gam <- gam(ndvi ~ s(altitude) + s(x, y), data =  
donnees) summary(modele_gam) plot(modele_gam)
```

`s(altitude)` ajuste une courbe lisse (spline) plutôt qu'une droite pour l'effet de l'altitude, dont la forme n'est pas imposée à l'avance mais estimée à partir des données elles-mêmes. `s(x, y)` ajoute un lissage bidimensionnel des coordonnées, qui capture une tendance spatiale résiduelle non expliquée par les variables explicatives — une alternative continue au découpage discret d'un modèle SAR/CAR. `plot(modele_gam)` trace chaque courbe lisse ajustée séparément.

EXEMPLE

QUAND PRÉFÉRER UN GAM À UNE RÉGRESSION SPATIALE

Un GAM s'impose quand le doute porte sur la forme de la relation elle-même (linéaire ou non), tandis que SAR/CAR s'imposent quand la relation est linéaire mais que ce sont les résidus qui restent spatialement structurés. Les deux logiques peuvent se combiner (un GAM avec un terme $s(x, y)$ joue en partie ce rôle), mais partent d'un diagnostic différent : non-linéarité contre autocorrélation résiduelle.

APPROFONDISSEMENT

5. SIMULATION MONTE CARLO, BOOTSTRAP SPATIAL ET TESTS DE PERMUTATION

`moran.test()` (section 1) est un test paramétrique : il suppose que l'indice de Moran suit, sous l'hypothèse nulle d'absence d'autocorrélation, une distribution théorique connue. Un test de permutation construit à la place cette distribution empiriquement, en recalculant l'indice sur des milliers de réarrangements aléatoires de la même donnée — sans supposer aucune forme théorique a priori.

MORAN.MC() : TEST DE MORAN PAR PERMUTATION (MONTE CARLO)

```
moran.mc(communes$taux, lw, nsim = 999)
```

`moran.mc()` recalcule l'indice de Moran sur $nsim = 999$ permutations aléatoires des valeurs de `communes$taux` entre les entités (la position géographique reste fixe, seules les valeurs sont mélangées), puis situe l'indice réellement observé dans cette distribution simulée. Une valeur observée extrême par rapport aux 999 simulations ($p \approx \text{rang} / (nsim + 1)$) signale une autocorrélation significative, sans dépendre d'une hypothèse de normalité sous-jacente comme `moran.test()`.

MORAN.TEST() — PARAMÉTRIQUE

- Suppose une distribution théorique connue sous l'hypothèse nulle
- Calcul immédiat, aucune simulation nécessaire
- Moins fiable si les hypothèses distributionnelles sont mal respectées (petits échantillons, valeurs extrêmes)

MORAN.MC() — PERMUTATION

- Construit la distribution empiriquement par simulation (nsim répétitions)
- Aucune hypothèse de normalité requise
- Plus coûteux en calcul, mais plus robuste sur données irrégulières

BOOTSTRAP MANUEL : RÉ-ÉCHANTILLONNER AVEC REMISE

```
moyennes_bootstrap <- replicate(1000,  
mean(sample(communes$taux, replace = TRUE)))  
quantile(moyennes_bootstrap, c(0.025, 0.975))
```

`sample(x, replace = TRUE)` tire un nouvel échantillon de même taille que `x`, avec remise (une même valeur peut être tirée plusieurs fois). `replicate(1000, ...)` répète l'opération 1000 fois et empile les résultats. `quantile(..., c(0.025, 0.975))` donne un intervalle de confiance empirique à 95 %, sans supposer de distribution théorique pour la moyenne — le même principe que `moran.mc()` ci-dessus, appliqué ici à une statistique non spatiale pour en montrer la généralité.

ATTENTION

UN BOOTSTRAP NAÏF IGNORE ENCORE LA DÉPENDANCE SPATIALE

`sample(communes$taux, replace = TRUE)` traite chaque commune comme un tirage indépendant, en ignorant sa position — exactement le biais de pseudoréplication déjà signalé dans le module Les Statistiques. Un bootstrap spatial correct (par blocs géographiques contigus plutôt que par entité isolée, ou en conservant la structure de voisinage à chaque ré-échantillonnage) est nécessaire dès que la donnée elle-même est spatialement autocorrélée, sous peine de sous-estimer l'incertitude réelle malgré l'apparence de rigueur d'une méthode de rééchantillonnage.

APPROFONDISSEMENT

6. REPRODUCTIBILITÉ : RENV, HERE, GIT

Un script qui fonctionne aujourd'hui peut cesser de fonctionner dans un an si les packages installés ont changé de version entre-temps (une fonction renommée, un comportement par défaut modifié). Trois pratiques garantissent qu'un projet R reste reproduc-

tible : verrouiller les versions de packages, ne jamais dépendre d'un chemin de fichier absolu, et suivre l'évolution du code avec Git.

RENV : VERROUILLER LES VERSIONS DE PACKAGES D'UN PROJET

```
renv::init() # démarre un environnement isolé pour ce projet
renv::snapshot() # enregistre les versions exactes des
packages utilisés dans renv.lock renv::restore() # réinstalle
exactement ces versions sur une autre machine
```

renv.lock (un fichier texte versionnable avec Git) fixe la version exacte de chaque package utilisé. Une autre personne (ou soi-même, un an plus tard) qui clone le projet et lance renv::restore() retrouve un environnement R identique, package par package — le même problème que résout requirements.txt en Python, ou renv.lock lui-même documente.

HERE::HERE() : DES CHEMINS RELATIFS AU PROJET, PAS À LA MACHINE

```
donnees <- read.csv(here::here('data', 'communes.csv'))
```

here::here() construit un chemin relatif à la racine du projet (repérée automatiquement via le fichier .Rproj ou un marqueur .here), plutôt qu'un chemin absolu (C:/Users/... ou /home/...) qui ne fonctionne que sur la machine où le script a été écrit. Un script qui utilise here::here() partout s'exécute à l'identique une fois le dossier du projet copié ou cloné ailleurs.

- renv verrouille les versions de packages (renv.lock), Git verrouille les versions du code lui-même : les deux se complètent, l'un sans l'autre laisse la reproductibilité incomplète
- RStudio intègre un panneau Git natif (onglet Git, en haut à droite) : commit, historique, diff visuel, sans quitter l'IDE
- Un .gitignore de projet R exclut typiquement .Rhistory, .RData et le dossier renv/library (les binaires de packages, régénérés par renv::restore(), n'ont pas leur place dans l'historique Git)

VOIR AUSSI : ALLER PLUS LOIN SUR GIT

Le module VS Code détaille Git en profondeur (branches, historique, résolution de conflits) au-delà du panneau Git intégré de RStudio, suffisant pour un usage courant mais plus limité.

APPROFONDISSEMENT

7. QUARTO ET R MARKDOWN AVANCÉ : RAPPORTS PARAMÉTRÉS

Quarto (successeur multi-langage de R Markdown, même principe de chunks exécutables mêlés à du texte) permet de paramétrer un rapport : plutôt que de rédiger un rapport par commune à la main, un unique fichier .qmd, écrit une fois, se recompile automatiquement pour chaque commune en substituant un paramètre.

PARAMÈTRES DÉCLARÉS DANS L'EN-TÊTE YAML D'UN .QMD

```
--- title: 'Bilan climatique' format: html params: region: 'Occitanie' --- Bilan pour la région `r params$region`.
```

params: region: 'Occitanie' déclare une valeur par défaut. Dans le corps du document, params\$region la réutilise partout où le rapport doit s'adapter à la région choisie (titre, filtre dplyr, sous-titre de graphique).

GÉNÉRER AUTOMATIQUEMENT UN RAPPORT PAR RÉGION

```
regions <- c('Occitanie', 'Bretagne', 'Grand Est') for (r in regions) { quarto::quarto_render('bilan.qmd', output_file = paste0('bilan_', r, '.html'), execute_params = list(region = r)) }
```

Une boucle for combinée à quarto_render() (package quarto) recompile bilan.qmd une fois par élément de regions, en changeant execute_params à chaque tour : trois rapports HTML distincts générés automatiquement, sans jamais rouvrir ni modifier le fichier source à la main — la même logique d'automatisation qu'un script PyQGIS/GeoPandas sur cinquante fichiers, appliquée ici à la production de rapports plutôt qu'à un traitement géographique.

EXEMPLE

LE MÊME PRINCIPE QU'UN MAILING, APPLIQUÉ À UN RAPPORT DE DONNÉES

Produire un rapport paramétré par commune, département ou année revient au même geste qu'un publipostage de courrier type : un seul gabarit, une liste de valeurs à substituer, autant de documents finaux générés que d'éléments dans la liste — sauf qu'ici chaque document recalcule aussi ses propres chiffres et graphiques à partir de la donnée réelle, pas seulement un nom et une adresse.

- Bilan — à retenir : `spdep` recalcule Moran en R (`poly2nb`, `nb2listw`, `moran.test`), matrice de voisinage à documenter comme tout choix méthodologique ; `spatialreg` corrige l'autocorrélation résiduelle qu'une OLS classique ignore (`lagsarlm`, `spautolm`), au prix d'une interprétation plus subtile des coefficients ; `gstat` implémente le krigeage de bout en bout (`variogram`, `fit.variogram`, `krige`), la contrepartie directe et reproductible d'un module d'interpolation graphique ; `mgcv::gam()` ajuste une relation non linéaire sans lui imposer de forme a priori ; `moran.mc()` et le bootstrap testent par simulation, sans hypothèse de distribution théorique, à condition de respecter la dépendance spatiale dans le rééchantillonnage ; `renv` verrouille les packages, `here::here()` les chemins, Git le code — les trois ensemble garantissent la reproductibilité ; Quarto paramétré automatise la production de rapports en substituant un paramètre à chaque recompilation.

VOIR AUSSI : PRATIQUER : KRIGEAGE ET INTERPOLATION EN CONDITIONS RÉELLES

L'Atelier applique le krigeage à un jeu de données radar/pluviométrique complet — de quoi comparer directement le résultat obtenu ici en R à celui obtenu par une interface graphique.